VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

Comparative Analysis of A* and Dijkstra Algorithms for Autonomous Mobile Robot Path Planning Using MATLAB

A project report submitted for the course
Robotics and Automation (BECE312L)

Bachelor of Technology in Electronics and Communication Engineering

by

Dhanya S (23BEC1432)
Nithujaa J (23BEC1354)
Sri Nithi Sathiyamoorthy (23BEC1440)

Faculty: Dr. Bharathi Sankar

School of Electronics Engineering
Vellore Institute of Technology, Chennai
Vandalur – Kelambakkam Road, Chennai – 600127, India

November 2026

Abstract

An autonomous mobile robot must find a collision-free route that is short and must find it quickly on limited onboard hardware. This report compares the two classical grid planners used for this task, Dijkstra's algorithm and A*, through a controlled benchmark built in MATLAB. The environment is modelled as an eight-connected occupancy grid with straight moves of cost 1, diagonal moves of cost √2 and no corner cutting. Both planners are implemented in one shared search function, so that A* differs from Dijkstra only by its heuristic. Forty-five seeded random maps were generated across three sizes (20 × 20, 50 × 50 and 100 × 100 cells) and three obstacle densities (10 %, 20 % and 30 %), and path length, planning time, nodes expanded and memory were recorded for every run. With an admissible heuristic, A* returned a path of exactly the same cost as Dijkstra on all 45 maps while expanding 36–78 % fewer nodes; on large open maps it was about three times faster. The advantage depended mainly on obstacle density. Among three heuristics, the Octile distance explored about 27 % fewer nodes than the Euclidean distance without losing optimality, whereas the Manhattan distance overestimated diagonal moves and gave a longer path on 76 % of the maps. A binary-heap open list made both planners roughly an order of magnitude faster with identical paths. The planned path was smoothed by line-of-sight shortcutting and followed to the goal by a simulated differential-drive robot under pure-pursuit control. The results are summarised in a scenario-based guideline for choosing between the two algorithms.

Keywords: path planning; A* algorithm; Dijkstra's algorithm; occupancy grid; heuristic search; admissibility; binary heap; pure pursuit; differential-drive robot; MATLAB.

Acknowledgements

We thank our course faculty and the School of Electronics Engineering, Vellore Institute of Technology, Chennai, for their guidance and feedback during the three reviews of this project in the Robotics and Automation course, and for the computing facilities we used to run the simulations.

We are also grateful to the authors of the textbooks and papers listed in the references, whose work on graph search and mobile robotics formed the basis of this study.

Finally, we thank our families and friends for their encouragement throughout the semester.

Contents

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Why Compare A* and Dijkstra? 1
1.3 Problem Statement 1
1.4 Objectives 2
1.5 Scope and Delimitations 2
1.6 Organisation of the Report 2
2 Literature Survey **3**
2.1 Shortest Paths in Graphs 3
2.2 Heuristic Search and A* 3
2.3 Grid Maps and Benchmarking 3
2.4 Faster and More Natural Grid Planners 3
2.5 From a Path to Motion 3
2.6 Gaps and Contribution of This Work 4
3 Theory and Background **5**
3.1 The Occupancy Grid and Eight-Connected Motion 5
3.2 Dijkstra's Algorithm 5
3.3 The A* Algorithm 5
3.4 Heuristics for an Eight-Connected Grid 6
3.5 The Open List and its Data Structure 6
3.6 Robot Motion Model and Path Tracking 7
4 System Design and Implementation **8**
4.1 Overall Architecture 8
4.2 Map Generation 8
4.3 The Unified Search Function 8
4.4 Performance Metrics 9
4.5 Experimental Design 10
4.6 Path Smoothing 10
4.7 Robot Simulation 10
4.8 Software Organisation and Dashboard 11
5 Results and Discussion **12**
5.1 Optimality Check 12
5.2 Search Effort and Planning Time 12
5.3 Effect of Map Size and Obstacle Density 13
5.4 Visualising the Search Space 14
5.5 Comparison of Heuristics 15
5.6 The Cost of an Inadmissible Heuristic 15
5.7 Effect of the Open-List Data Structure 16
5.8 Robot Following the Planned Path 16
5.9 Decision Guideline 17
5.10 Strengths and Limitations 18
5.11 Industrial Relevance 19
6 Conclusion and Future Scope **20**
6.1 Conclusion 20
6.2 Future Scope 20
References **21**

List of Figures

Figure 3.1 The motion model: (a) the eight allowed moves with their costs, (b) a diagonal move between two touching obstacles is blocked.....5
Figure 4.1 System architecture: the same map, start and goal are passed to every planner through one shared search function.....8
Figure 4.2 Flowchart of the shared search loop. With h = 0 the same loop is Dijkstra's algorithm.....9
Figure 5.1 Nodes expanded (top) and planning time (bottom) against map size for each obstacle density (logarithmic vertical axes).....13
Figure 5.2 Node reduction (left) and speed-up (right) of A* over Dijkstra for all nine configurations.....14
Figure 5.3 Explored cells on medium-density maps: light blue = Dijkstra only, orange = both. Pink and green lines are the Dijkstra and A* paths.....14
Figure 5.4 Percentage of nodes saved relative to Dijkstra by A* with the Euclidean, Octile and Manhattan heuristics.....15
Figure 5.5 A* with the Manhattan heuristic: mean path excess over the shortest path (left) and percentage of maps with a non-optimal path (right).....15
Figure 5.6 Planning time on 100 × 100 maps with a linear-scan and a binary-heap open list (logarithmic scale).....16
Figure 5.7 Robot motion: raw and smoothed paths with the driven trajectory (left), wheel speeds (top right) and distance from the planned path (bottom right).....17
Figure 5.8 Final frame of the animation: the trajectory of the differential-drive robot from start (blue) to goal (red).....17
Figure 5.9 Decision guideline for choosing between Dijkstra and A* (Octile heuristic, binary-heap open list).....18

List of Tables

Table 2.1 Comparison of common grid path-planning approaches.....4
Table 3.1 Heuristics compared in this project.....6
Table 4.1 Metrics recorded for every run.....9
Table 4.2 Experimental configuration.....10
Table 4.3 Parameters of the robot simulation.....10
Table 5.1 Benchmark results: Dijkstra vs A* (Euclidean), mean of 5 maps per configuration.....12
Table 5.2 Industrial and research areas related to the project.....19

List of Acronyms

Acronym	Full form
A*	A-star search algorithm
AGV	Automated Guided Vehicle
CSV	Comma-Separated Values
D* Lite	Dynamic A* (incremental replanning algorithm)
DWA	Dynamic Window Approach
HPA*	Hierarchical Path-finding A*
JPS	Jump Point Search
PNG	Portable Network Graphics
ROS	Robot Operating System
RRT*	Optimal Rapidly-exploring Random Tree
SLAM	Simultaneous Localisation and Mapping

CHAPTER 1
Introduction

1.1 Background and Motivation

An autonomous mobile robot has to answer one question many times a day: how do I get from where I am to where I need to be without hitting anything? A warehouse carrier moving a shelf, a delivery robot in a hospital corridor and a cleaning robot under a sofa all face the same problem. It is called *path planning*, and it sits between perception, which tells the robot what the world looks like, and control, which turns a plan into wheel commands [1].

Two requirements pull against each other. The path should be as short as possible, because every extra metre costs battery charge and time on every trip. The path must also be found quickly, because the robot runs on a small onboard computer and may have to replan whenever a person steps into its way. An algorithm that is perfect but slow is as unhelpful as one that is fast but wasteful [2].

Most indoor robots describe their surroundings as a grid of small square cells, each marked free or occupied. Planning on such a grid turns into a search through a graph, and two classical algorithms dominate that search: Dijkstra's algorithm, published in 1959 [3], and A* (read "A-star"), introduced in 1968 [4]. Both are taught in every robotics course, and both appear in modern software such as the navigation stack of the Robot Operating System [5].

1.2 Why Compare A* and Dijkstra?

Dijkstra's algorithm grows outward from the start in every direction at once, a little like ripples spreading on a pond, until the ripple touches the goal. It always finds the shortest path, but much of that effort is spent on cells that lie behind the robot or off to the side. A* uses the same mechanism, but it adds an estimate of the remaining distance to the goal, called a *heuristic*. The estimate tilts the search toward the goal so that fewer cells are examined.

Textbooks state that A* is "faster", but the statement hides several questions that matter to an engineer choosing a planner:

- **How much faster?** Is the saving ten percent or ninety percent, and does it change with the size of the map?
- **When does it matter?** On a small map both algorithms may finish in well under a millisecond, in which case the difference is irrelevant.
- **Which heuristic?** There are several common distance estimates, and not all of them preserve the guarantee of a shortest path.
- **Does the implementation change the answer?** The data structure that stores the cells waiting to be examined can dominate the running time.

1.3 Problem Statement

Published comparisons of A* and Dijkstra often test a single hand-drawn map, report a single number such as running time, and rarely explain why the numbers come out as they do. A designer who reads them cannot tell whether the conclusion would hold on a larger floor, a more cluttered room, or with a different heuristic.

This project therefore asks: *how much better is A* than Dijkstra for grid-based mobile robot path planning, under which conditions, and which version of A* should a robot use?* The question is answered with a controlled, repeatable benchmark in MATLAB rather than with a single example.

1.4 Objectives

1. To model the robot's environment as a two-dimensional occupancy grid with eight-connected motion and to forbid diagonal moves that squeeze between two obstacles.
2. To implement Dijkstra's algorithm and A* in one shared search function so that the only difference between them is the heuristic.
3. To benchmark both planners on 45 seeded random maps covering three map sizes and three obstacle densities, logging path length, planning time, nodes expanded and memory for every run.
4. To verify on every map that A* returns a path exactly as short as Dijkstra's, and to compare the Euclidean, Octile and Manhattan heuristics.
5. To measure the effect of replacing a linear-scan open list by a binary heap.
6. To show that the planned path can be followed by a simulated differential-drive robot after smoothing, and to turn all results into a scenario-based decision guideline.

1.5 Scope and Delimitations

The work is a simulation study. The maps are static and fully known before planning begins, the robot is treated as occupying one cell, and obstacles are randomly scattered single cells rather than walls or furniture. Timing figures were measured in MATLAB on a laptop and are useful for comparing the two planners with each other, not as absolute predictions for an embedded processor. Dynamic obstacles, sensing errors and replanning during motion are outside the scope and are listed as future work.

1.6 Organisation of the Report

Chapter 2 reviews earlier work on graph search, heuristics and grid planning. Chapter 3 explains the theory behind the grid model, the two algorithms, the heuristics, the open list and the robot model. Chapter 4 describes how the benchmark and the robot simulation were built. Chapter 5 presents and discusses the results, and Chapter 6 concludes and outlines future work.

CHAPTER 2
Literature Survey

2.1 Shortest Paths in Graphs

Dijkstra's 1959 note solved the single-source shortest-path problem for graphs whose edge costs are non-negative [3]. Its key idea is that once the cheapest unvisited node has been selected, no later discovery can make its cost smaller, so it can be fixed permanently. The algorithm is simple and exact, and its running time depends mainly on how the next cheapest node is selected. Later textbooks showed that a priority queue such as a binary heap reduces this cost from a scan of every node to a logarithmic operation [6], [7].

Because Dijkstra's algorithm computes the cost of reaching every node, one run answers many questions at once. That property is still exploited in robotics, for example when a robot must pick the nearest of several charging stations, but it also explains the algorithm's weakness when only one goal matters.

2.2 Heuristic Search and A*

Hart, Nilsson and Raphael introduced A* while working on the Shakey robot at the Stanford Research Institute [4]. They ranked each node by the sum of its known cost from the start and an estimate of the cost still to go, and proved that A* returns an optimal path whenever the estimate never exceeds the true remaining cost. Such an estimate is called *admissible*. Pearl's book later gave a thorough account of admissibility, consistency and the trade-off between the accuracy of a heuristic and the work it saves [8].

The same framework allows deliberate compromise. Pohl's weighted A* multiplies the heuristic by a factor larger than one, which speeds up the search but gives up the guarantee of the shortest path [9]. An inadmissible heuristic chosen by accident has exactly the same effect, and Chapter 5 shows that this happens with the Manhattan distance on an eight-connected grid.

2.3 Grid Maps and Benchmarking

Elfes popularised the occupancy grid as a way for a mobile robot to store what its sensors have seen, with each cell holding the probability that it is occupied [10]. Thresholding that probability gives the binary grid used in this project. Thrun, Burgard and Fox describe how such grids are built and maintained in practice [11].

For comparing planners, Sturtevant argued that results are only meaningful when they are measured on many maps with controlled properties, and he released standard benchmark sets for grid pathfinding [12]. Our multi-scenario design follows the same spirit on a smaller scale: instead of one map we use nine configurations and five random maps in each.

2.4 Faster and More Natural Grid Planners

Several refinements reduce the work further or improve the shape of the path. Jump Point Search skips long runs of symmetric cells on uniform-cost grids [13]. Hierarchical methods plan first on a coarse grid and then refine [14]. Theta* allows the path to leave the grid lines, which removes many of the zig-zags that an eight-connected planner produces [15]. D* Lite repairs a previous plan when the map changes rather than starting again [16]. Sampling-based planners such as RRT* are preferred in high-dimensional spaces such as robot arms [17]. These methods build on the A* framework and are natural extensions of the present work.

2.5 From a Path to Motion

A grid path is a list of cell centres joined by straight and diagonal steps, and a wheeled robot cannot follow its sharp corners exactly. Coulter's pure-pursuit method, developed for autonomous land vehicles at Carnegie Mellon University, steers the robot toward a point a fixed

distance ahead on the path and remains one of the most widely used path trackers [18]. Local planners such as the dynamic window approach handle obstacles that appear during motion [19].

Table 2.1: Comparison of common grid path-planning approaches

Method	Shortest path?	Search effort	Typical remark
Dijkstra	Yes	High: expands outward in all directions	Gives costs to every cell; simple and robust
A* (admissible heuristic)	Yes	Lower: focused toward the goal	Benefit depends on map and heuristic
Weighted A*	No (bounded)	Lower still	Trades path quality for speed
Jump Point Search	Yes	Much lower on open grids	Needs uniform-cost grids
Theta*	Near-shortest	Similar to A*	Any-angle, smoother paths
D* Lite	Yes	Low on replanning	Designed for maps that change

2.6 Gaps and Contribution of This Work

The surveyed work provides the theory, but three practical gaps remain for a student or engineer choosing between the two classical planners. First, many comparisons rely on a single map and a single metric. Second, the effect of the heuristic is usually discussed in theory but not measured, so the cost of an inadmissible heuristic is rarely quantified. Third, the conclusion is seldom turned into a rule that says when the choice of algorithm actually matters.

This project contributes a reproducible MATLAB benchmark of 45 seeded maps in which both planners share the same code, four metrics logged on every run, an optimality check on every map, a measured comparison of three heuristics and two open-list structures, a visual comparison of the cells each algorithm explores, a simulation showing that the plan can be driven, and a decision guideline derived from the measurements.

CHAPTER 3
Theory and Background

3.1 The Occupancy Grid and Eight-Connected Motion

The floor is divided into an N × N grid. Each cell is either free or occupied, and the robot is assumed to fit inside one cell. From any free cell the robot may move to its eight neighbours. A straight move costs 1 and a diagonal move costs √2 ≈ 1.414, which is the true distance between the cell centres (Figure 3.1a). The grid is therefore a weighted graph whose nodes are free cells and whose edges are the allowed moves.

One detail matters for a physical robot. If two obstacles touch only at a corner, a diagonal move between them would pass through a gap of zero width. Our planner forbids such a move: a diagonal step is allowed only if both of the straight cells beside it are free (Figure 3.1b). This rule makes every planned path physically passable and is applied identically to both algorithms.

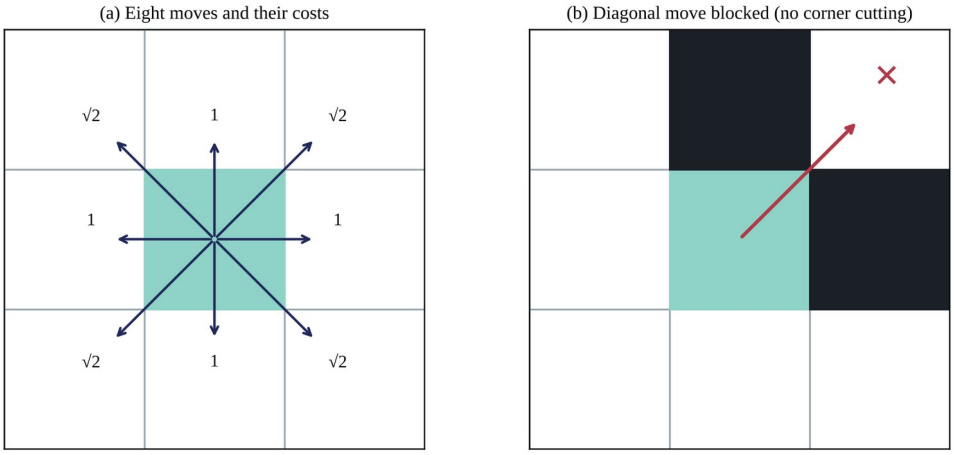


Figure 3.1: The motion model: (a) the eight allowed moves with their costs, (b) a diagonal move between two touching obstacles is blocked.

3.2 Dijkstra's Algorithm

Dijkstra's algorithm keeps, for every cell n, the cost g(n) of the cheapest path found so far from the start. Cells that have been reached but not yet finalised are held in an *open list*. At each step the algorithm removes the open cell with the smallest g, marks it *closed*, and examines its neighbours. If the route through the current cell gives a neighbour a smaller cost than it had before, the neighbour's cost and its parent pointer are updated:

$$g(m) \leftarrow \min\{\, g(m),\ g(n) + c(n, m) \,\} \tag{1}$$

where c(n, m) is 1 or √2. The search stops when the goal is removed from the open list, and the path is recovered by following parent pointers back to the start. Because cells are finalised in increasing order of cost, the first time the goal is closed its cost is the minimum possible. The price is that every cell cheaper than the goal is closed first, and on an open map that is most of the map.

3.3 The A* Algorithm

A* changes only the rule used to pick the next cell. Instead of the smallest g(n), it selects the smallest

$$f(n) = g(n) + h(n) \tag{2}$$

where h(n) is the heuristic estimate of the cost from n to the goal. Cells that lie in the direction of the goal receive a small h and are explored early; cells that lie behind the start receive a large h

and are often never explored at all. Setting h(n) = 0 for every cell turns A* back into Dijkstra's algorithm. This is how both planners were implemented in this project, so that any difference in the results comes from the heuristic and from nothing else.

Two properties of the heuristic decide the behaviour of A* [4], [8]. A heuristic is *admissible* if it never overestimates:

$$h(n) \le h^*(n) \quad \text{for every cell } n \tag{3}$$

where h*(n) is the true cheapest cost from n to the goal. With an admissible heuristic A* is guaranteed to return a shortest path. A heuristic is *consistent* if $h(n) \le c(n, m) + h(m)$ for every move from n to m; consistency implies admissibility and ensures that no cell has to be reopened after it is closed. Among admissible heuristics, the one closer to h* explores fewer cells.

3.4 Heuristics for an Eight-Connected Grid

Let Δx and Δy be the horizontal and vertical distances, in cells, from a cell to the goal. Three distance estimates are compared in this work.

- **Euclidean:** $h = \sqrt{(\Delta x^2 + \Delta y^2)}$, the straight-line distance. It is admissible because no route can be shorter than a straight line, but it underestimates on a grid, where the robot cannot move at arbitrary angles.
- **Octile:** $h = \max(\Delta x, \Delta y) + (\sqrt{2} - 1)\cdot\min(\Delta x, \Delta y)$. It is the exact cost of the cheapest route on an empty eight-connected grid: diagonal steps until one coordinate matches, then straight steps. It is admissible and is the tightest of the three.
- **Manhattan:** $h = \Delta x + \Delta y$, the cost when only horizontal and vertical moves are allowed. On an eight-connected grid it counts one diagonal step as 2 instead of √2, so it overestimates and is not admissible.

Table 3.1: Heuristics compared in this project

Heuristic	Formula	Admissible here?	Expected effect
None (Dijkstra)	h = 0	Yes	Shortest path, most cells explored
Euclidean	$\sqrt{(\Delta x^2 + \Delta y^2)}$	Yes	Shortest path, fewer cells
Octile	$\max(\Delta x,\Delta y) + (\sqrt{2}-1)\cdot\min(\Delta x,\Delta y)$	Yes	Shortest path, fewest cells of the admissible set
Manhattan	$\Delta x + \Delta y$	No	Very few cells, but path may be longer

3.5 The Open List and its Data Structure

Both algorithms repeatedly ask the open list for its cheapest cell. The simplest implementation stores a cost for every cell of the map and scans the whole array to find the minimum. Each removal then costs time proportional to the number of cells N^2, and a full search costs roughly N^4 operations.

A *binary heap* keeps the open cells in a tree in which every parent is cheaper than its children, so the cheapest cell is always at the root [7], [20]. Removing it or inserting a new cell costs time proportional to the logarithm of the heap size. When a cell's cost improves, our implementation simply inserts a new copy and ignores outdated copies when they are removed later, a technique known as lazy deletion. The heap changes only the speed of the search; the order in which cells are finalised, and therefore the path cost, is unchanged apart from ties between cells of equal cost.

3.6 Robot Motion Model and Path Tracking

The robot is modelled as a differential-drive vehicle with two independently driven wheels separated by a distance b. If v is its forward speed and ω its turning rate, the pose (x, y, θ) evolves as

$$\dot{x} = v \cos\theta, \quad \dot{y} = v \sin\theta, \quad \dot{\theta} = \omega \tag{4}$$

and the wheel speeds are $v_R = v + \omega b/2$ and $v_L = v - \omega b/2$. The robot turns by running one wheel faster than the other.

Pure pursuit [18] chooses ω by aiming at a *lookahead point* on the path at a distance L ahead of the robot. If α is the angle between the robot's heading and the line to that point, the arc that reaches the point has curvature $2 \sin\alpha / L$, so the turning rate is

$$\omega = 2 v \sin\alpha / L \tag{5}$$

A short lookahead follows the path closely but reacts nervously; a long one is smooth but cuts corners. Because a grid path has a corner at almost every cell, the path is first simplified by line-of-sight shortcutting, described in Section 4.6, before it is given to the tracker.

CHAPTER 4
System Design and Implementation

4.1 Overall Architecture

Figure 4.1 shows the structure of the software. A configuration block defines the experiment, a generator produces a reproducible map for each case, and a single search function serves every planner. Results are logged, checked against Dijkstra's optimal cost and written out as tables and figures. In parallel, the Octile path on a chosen map is smoothed and driven by the simulated robot. Everything runs in core MATLAB without any toolbox.

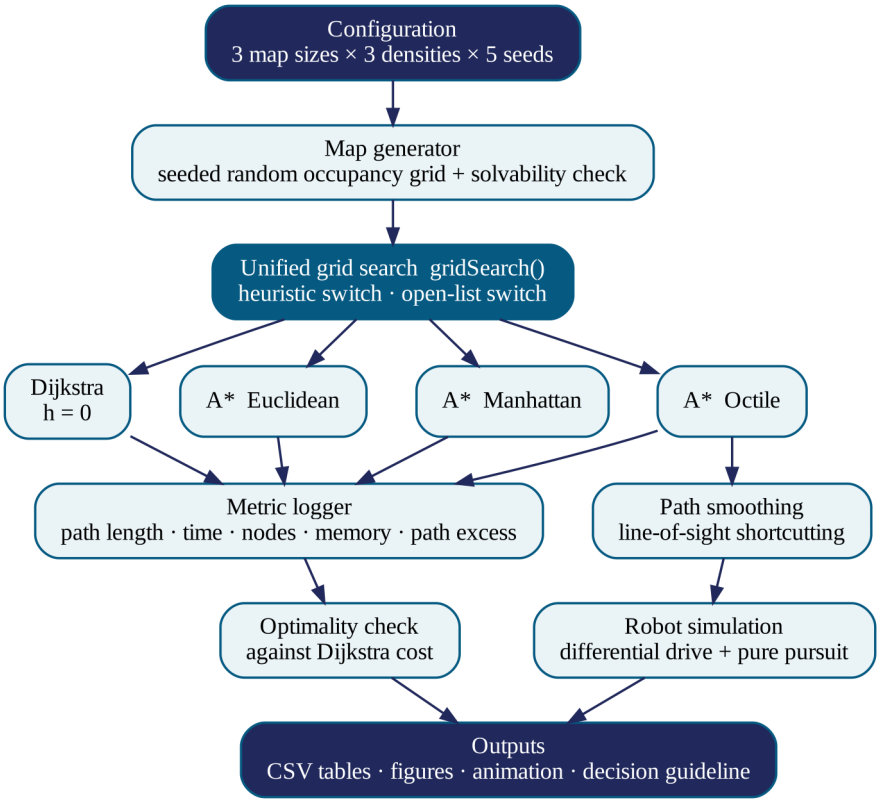


Figure 4.1: System architecture: the same map, start and goal are passed to every planner through one shared search function.

4.2 Map Generation

Each map is an N × N logical matrix in which a cell is marked occupied with probability equal to the chosen obstacle density. The random-number generator is seeded with a value derived from the map size, the density and the trial number, so every map can be regenerated exactly by anyone who runs the script. The start is fixed at cell (2, 2) and the goal at (N − 1, N − 1), and the 3 × 3 block around each is cleared so that neither is boxed in. Dijkstra's algorithm is then run once to confirm that a path exists; if it does not, a new map is drawn with a shifted seed.

Fixing the start and goal at opposite corners makes every test a long, diagonal trip across the map, which is the hardest case for an unguided search and the clearest test of the heuristic.

4.3 The Unified Search Function

A single function, gridSearch, implements both planners. It takes the map, the start, the goal, the heuristic type (none, Euclidean, Octile or Manhattan) and the open-list type (linear scan or binary heap). The heuristic for every cell is computed once as a matrix before the search starts, so evaluating it inside the loop costs a single lookup. Figure 4.2 shows the logic of the search loop.

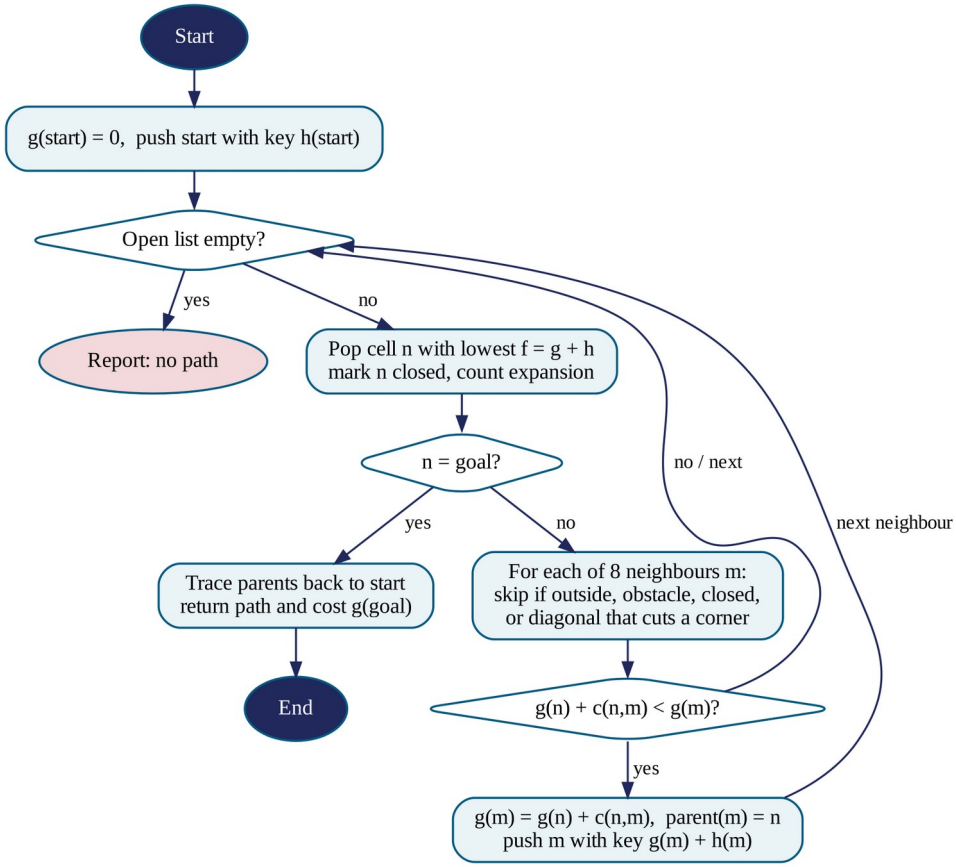


Figure 4.2: Flowchart of the shared search loop. With h = 0 the same loop is Dijkstra's algorithm.

Sharing one function was a deliberate design choice. If Dijkstra and A* were written separately, a difference in running time could come from a small difference in coding style rather than from the algorithms themselves. With one function, the comparison is fair by construction.

4.4 Performance Metrics

Five quantities were recorded for every planner on every map.

Table 4.1: Metrics recorded for every run

Metric	Definition	Why it matters
Path length	Cost g(goal): 1 per straight step, √2 per diagonal step	Distance the robot must travel
Planning time	Median of three repeated runs, measured with tic/toc	Responsiveness of the planner
Nodes expanded	Number of cells removed from the open list and closed	Work done; independent of the computer
Memory	Cells that ever received a cost × bytes per cell, plus peak open-list size	Storage needed during the search
Path excess	100 × (length / Dijkstra length − 1), in percent	Loss of optimality, if any

Nodes expanded is the most reliable measure of efficiency because it does not depend on the computer, the MATLAB version or background tasks. Timing is reported as well, because it is what a user feels, but sub-millisecond times are treated with caution.

4.5 Experimental Design

The benchmark was run in two stages. The first stage compared Dijkstra with A* using the Euclidean heuristic and a linear-scan open list, the configuration originally proposed for the project. The second stage reran the same 45 maps, with the same seeds, for all four planners using the binary-heap open list, and timed both open-list versions on the 100 × 100 maps.

Table 4.2: Experimental configuration

Parameter	Value
Map sizes	20 × 20, 50 × 50, 100 × 100 cells
Obstacle densities	Sparse 10 %, medium 20 %, dense 30 %
Maps per configuration	5 (seeded, reproducible) – 45 maps in total
Start and goal	(2, 2) and (N − 1, N − 1), neighbourhoods kept free
Motion model	8-connected, costs 1 and √2, no corner cutting
Planners	Dijkstra; A* with Euclidean, Octile and Manhattan heuristics
Open list	Linear scan (stage 1) and binary heap with lazy deletion (stage 2)
Timing	Median of 3 repeated runs per planner and map
Software	MATLAB, core functions only (no toolboxes)

4.6 Path Smoothing

The raw path from the planner follows cell centres and changes direction frequently. It was simplified by line-of-sight shortcutting: starting at the first waypoint, the algorithm looks for the farthest later waypoint that can be reached in a straight line without touching an obstacle, jumps to it, and repeats. To keep the robot away from obstacle edges, the straight segment is checked not only along its centre line but also along two parallel lines offset by 0.35 cell on either side. The result is a short list of waypoints joined by obstacle-free straight segments.

4.7 Robot Simulation

The smoothed path was resampled every 0.1 cell and followed by the differential-drive model of Equation (4) using the pure-pursuit law of Equation (5). The simulation integrates the motion with a fixed time step and records the pose, the wheel speeds, the distance from the planned path (tracking error), the clearance to the nearest obstacle and whether the robot ever enters an occupied cell. The robot slows down in proportion to its distance from the goal over the final lookahead distance and stops when it is within 0.3 cell of the goal. For comparison, the raw, unsmoothed path was driven with the same settings.

Table 4.3: Parameters of the robot simulation

Parameter	Value
Test map	50 × 50, medium density (20 %), first seed
Cruise speed v	1.0 cell/s
Lookahead distance L	1.5 cells
Maximum turning rate	2.5 rad/s
Wheel separation b	0.5 cell
Time step	0.05 s
Goal tolerance	0.3 cell
Smoothing clearance	± 0.35 cell

4.8 Software Organisation and Dashboard

The work is organised into three MATLAB scripts. The proof-of-concept script runs both planners on one hand-built 20 × 20 grid. The benchmark script generates the 45 maps, runs Dijkstra and Euclidean A*, checks optimality and produces the scaling plots, the reduction heatmaps and the search-space overlays. The final script adds the heuristic comparison, the open-list comparison, path smoothing, the robot simulation and animation, and the decision guideline. All tables are written as CSV files and all figures as PNG images.

To make the results easy to explore during the review, the same algorithms were also implemented in an offline browser dashboard. It reruns the benchmark when it is opened, lets the user draw walls and move the start and goal, and shows Dijkstra and A* searching the same map side by side. It needs no internet connection or installation.

CHAPTER 5
Results and Discussion

All numbers in this chapter are means over the five maps of each configuration, unless a single map is named. Sections 5.1 to 5.4 come from the first stage of the benchmark (Euclidean A*, linear open list) and Sections 5.5 to 5.9 from the second stage (all heuristics, binary heap).

5.1 Optimality Check

The first result is a check rather than a measurement. On every one of the 45 maps, A* with the Euclidean heuristic returned a path whose cost was identical to Dijkstra's. This confirms both the theory of Section 3.3 and the correctness of the implementation, and it means that every saving reported below comes at no cost in path quality.

The path lengths themselves behave sensibly. The straight diagonal from (2, 2) to (N − 1, N − 1) is the shortest conceivable route: 24.0 cells on the 20 × 20 map, 66.5 on the 50 × 50 map and 137.2 on the 100 × 100 map. The sparse-map paths are only 5–7 % longer than these bounds, and the paths grow longer as density increases, because the robot must make more detours around obstacles.

Table 5.1: Benchmark results: Dijkstra vs A (Euclidean), mean of 5 maps per configuration*

Map	Density	Dijkstra nodes	A* nodes	Node reduction	Dijkstra (ms)	A* (ms)	Speed-up	Memory reduction	Path length
20×20	Sparse	357	80	77.5 %	0.77	0.40	1.93×	61.7 %	25.68
20×20	Medium	320	166	48.3 %	0.44	0.32	1.39×	36.6 %	28.26
20×20	Dense	271	160	40.9 %	0.32	0.21	1.53×	32.5 %	32.16
50×50	Sparse	2248	634	71.8 %	5.00	1.64	3.05×	65.0 %	70.92
50×50	Medium	1987	933	53.1 %	3.95	1.95	2.03×	47.2 %	77.23
50×50	Dense	1683	1070	36.4 %	3.42	2.26	1.52×	32.3 %	86.57
100×100	Sparse	9013	2824	68.7 %	58.10	18.09	3.21×	65.1 %	145.97
100×100	Medium	7983	3945	50.6 %	51.41	25.42	2.02×	47.8 %	155.57
100×100	Dense	6858	4187	38.9 %	44.31	27.08	1.64×	36.6 %	172.04

5.2 Search Effort and Planning Time

Table 5.1 and Figure 5.1 show that A* expanded fewer nodes than Dijkstra in all nine configurations, by between 36 % and 78 %. Both algorithms need more work as the map grows, but the gap between them stays roughly constant on the logarithmic axes, which means A* is better by a roughly constant *factor* rather than a constant amount.

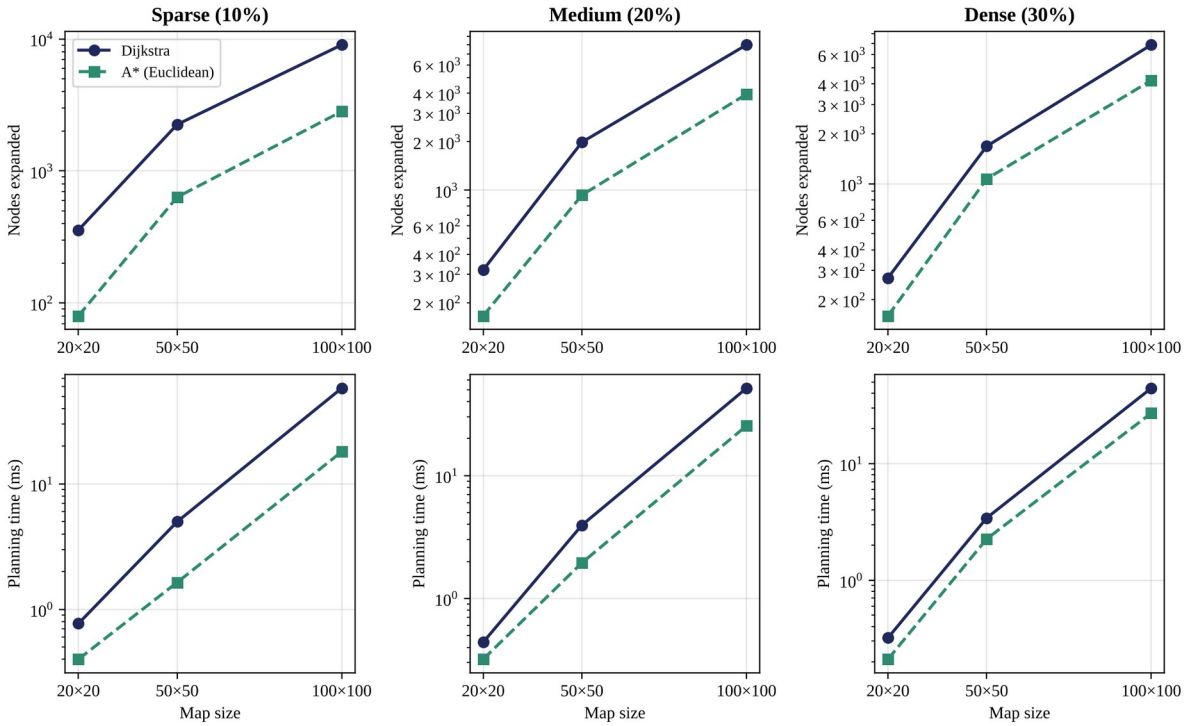


Figure 5.1: Nodes expanded (top) and planning time (bottom) against map size for each obstacle density (logarithmic vertical axes).

One observation stands out. On the 100 × 100 sparse maps about 9,000 cells are free, and Dijkstra expanded 9,013 of them. On the dense maps about 7,000 cells are free and Dijkstra expanded 6,858. In other words, Dijkstra searched almost the entire reachable map before it reached the opposite corner. A* searched less than a third of it on the sparse maps.

The memory reduction (32–65 %) is consistently smaller than the node reduction. This is expected rather than an error: the memory figure counts every cell that ever received a cost, including the cells still waiting in the open list when the goal is reached. A* stops early and therefore leaves a relatively larger frontier of reached but unexpanded cells.

5.3 Effect of Map Size and Obstacle Density

Figure 5.2 places all nine configurations side by side. The node reduction depends mainly on obstacle density: about 70–78 % on sparse maps, about 48–53 % on medium maps and about 36–41 % on dense maps. The reason is the quality of the estimate. On an open map the straight-line distance is close to the real remaining cost, so A* walks almost directly toward the goal. On a cluttered map obstacles force detours that the estimate cannot foresee, and A* has to explore sideways, much as Dijkstra does.

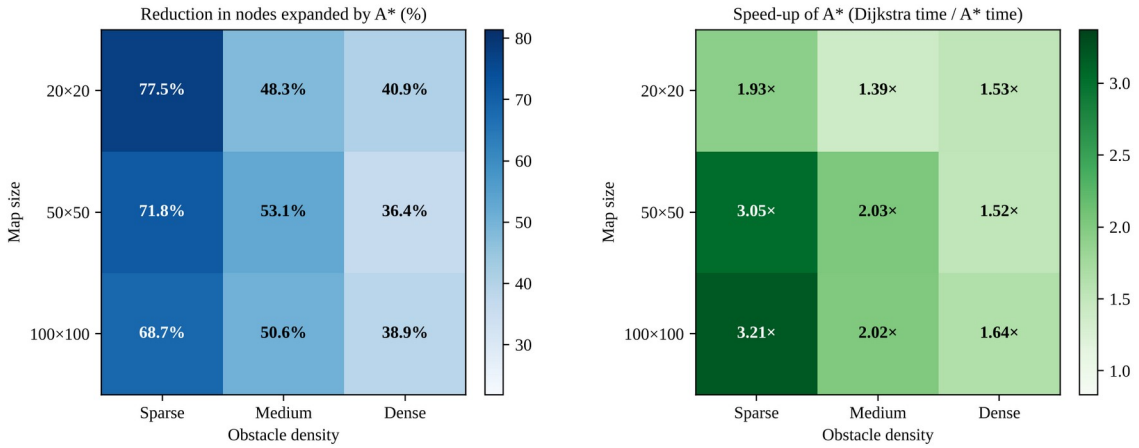


Figure 5.2: Node reduction (left) and speed-up (right) of A over Dijkstra for all nine configurations.*

The speed-up on the 100 × 100 maps agrees closely with the node reduction. If A* does a fraction (1 − r) of Dijkstra's work, it should be about 1/(1 − r) times faster. For a reduction r of 68.7 %, 50.6 % and 38.9 % this predicts speed-ups of 3.19×, 2.02× and 1.64×; the measured values were 3.21×, 2.02× and 1.64×. The agreement shows that the time per expansion is the same for both planners and that the timing results are consistent with the work done.

On the 20 × 20 maps the agreement breaks down: the node reduction is 77.5 % on sparse maps, but the speed-up is only 1.93×. Every run there takes less than a millisecond, so fixed costs such as allocating arrays and computing the heuristic matrix dominate. At this size we rely on node counts rather than on time.

5.4 Visualising the Search Space

Figure 5.3 shows, for one medium-density map of each size, which cells each algorithm explored. Light blue cells were explored by Dijkstra only and orange cells by both algorithms.

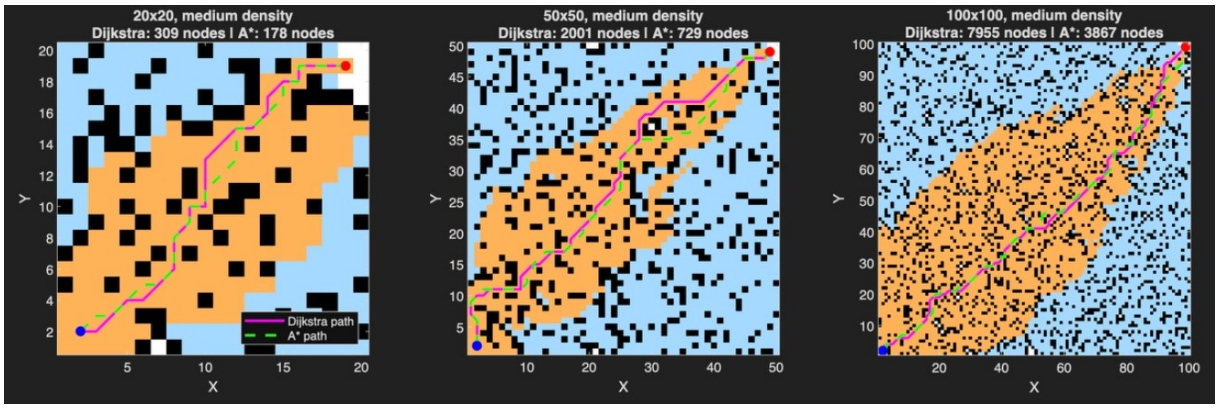


Figure 5.3: Explored cells on medium-density maps: light blue = Dijkstra only, orange = both. Pink and green lines are the Dijkstra and A paths.*

The picture explains the numbers. Dijkstra's search spreads into the corners of the map that lie away from the goal, and all of the blue area is wasted effort. A* stays inside a band along the diagonal toward the goal. No cell was explored by A* alone, so A*'s search was a strict subset of Dijkstra's on these maps, as expected for a consistent heuristic. The two paths sometimes follow different routes, but their lengths are equal: a grid usually contains several different routes of exactly the same cost.

5.5 Comparison of Heuristics

Figure 5.4 compares the three heuristics in the second stage of the benchmark. Octile saved more nodes than Euclidean in every configuration. Averaged over all 45 maps, A* with Octile expanded 1,134 nodes against 1,555 with Euclidean, about 27 % fewer, and both retained the shortest path. The improvement follows directly from Section 3.4: Octile is the exact cost on an empty eight-connected grid, so it is a tighter estimate than the straight line.

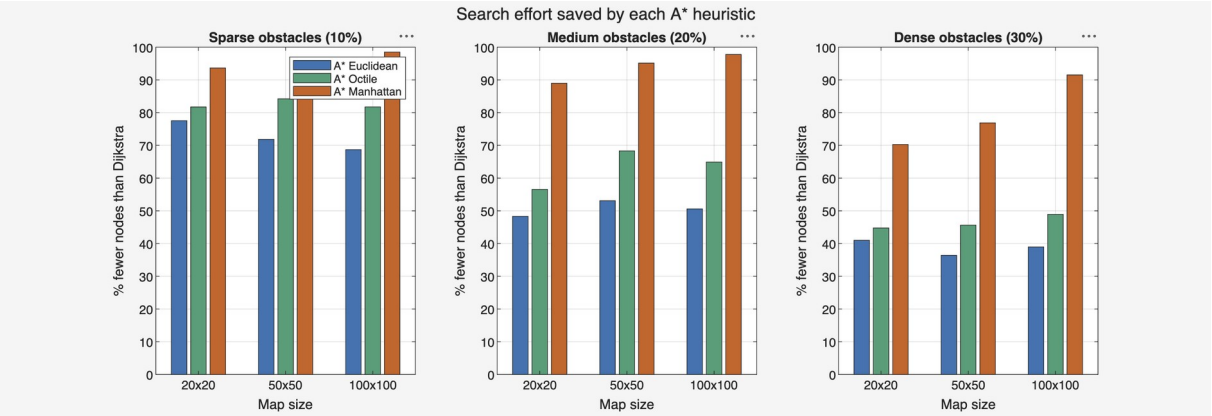


Figure 5.4: Percentage of nodes saved relative to Dijkstra by A with the Euclidean, Octile and Manhattan heuristics.*

The density trend seen in Section 5.3 holds for every heuristic. With Octile, the saving falls from 82–84 % on sparse maps to 45–49 % on dense maps. Manhattan appears to be the clear winner, saving up to about 99 % on sparse maps, but the next section shows why that figure is misleading.

5.6 The Cost of an Inadmissible Heuristic

Manhattan distance counts a diagonal step as 2 when its real cost is √2, so it overestimates the remaining cost and breaks the admissibility condition of Equation (3). A* then becomes over-confident: it follows the estimate too eagerly and can settle on a route that is not the shortest.

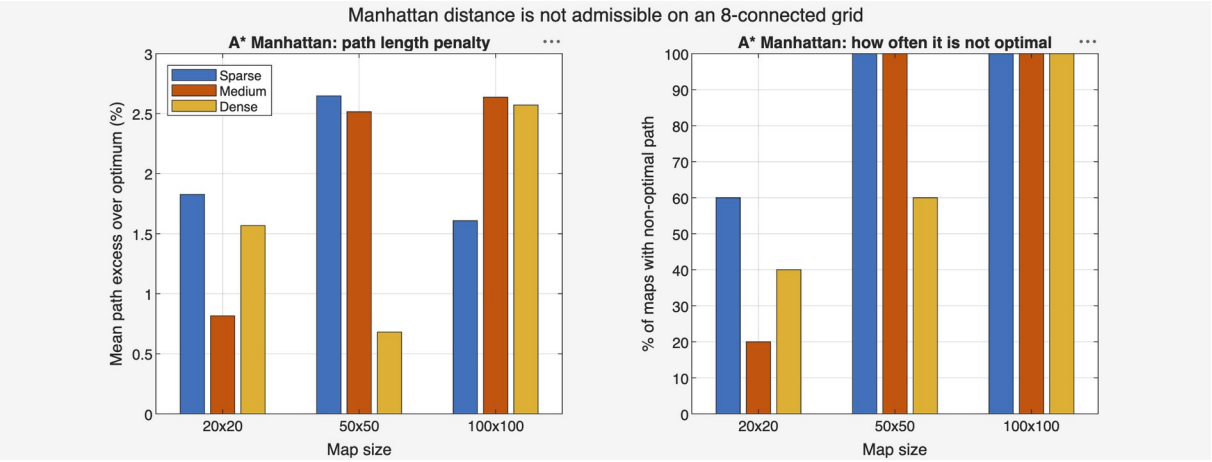


Figure 5.5: A with the Manhattan heuristic: mean path excess over the shortest path (left) and percentage of maps with a non-optimal path (right).*

Figure 5.5 measures the effect. A* with Manhattan returned a longer path than the optimum on 76 % of the maps (34 of 45). On large maps this happened on nearly every map, and the mean excess per configuration ranged from about 0.7 % to 2.7 %, with a worst case of 5.84 % on a single map. A few percent may look harmless, but it is paid on every trip the robot makes, whereas the time saved in planning is paid once and is already measured in milliseconds.

The conclusion is that Manhattan distance is the right heuristic only for robots that move in four directions, where it is exact. For an eight-connected grid Octile is the correct choice: it is admissible and it explores the fewest cells of the heuristics that keep the shortest path. This also corrects an assumption made at the start of the project, when Manhattan was listed alongside Euclidean as an admissible option.

5.7 Effect of the Open-List Data Structure

Figure 5.6 compares the planning time of the linear-scan and binary-heap open lists on the 100 × 100 maps. Both versions gave identical path costs on every map; only the speed changed.

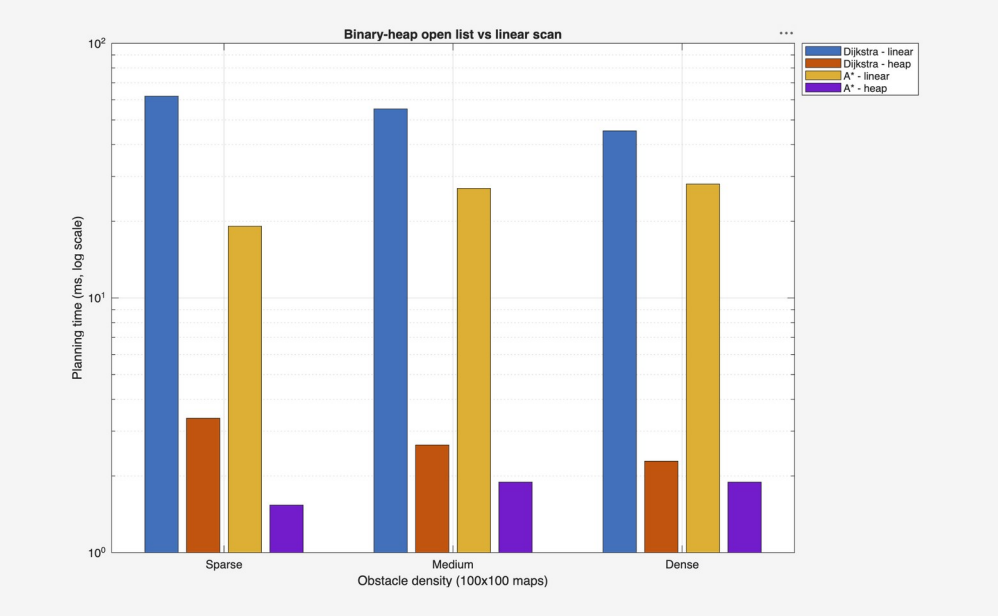


Figure 5.6: Planning time on 100 × 100 maps with a linear-scan and a binary-heap open list (logarithmic scale).

The heap made Dijkstra roughly 20 times faster, from about 45–60 ms to about 2–3 ms, and made A* roughly 13–15 times faster, from about 19–28 ms to about 2 ms. A* stayed faster than Dijkstra with both structures, so the main conclusion of the study does not depend on the implementation. The size of the gain, however, shows that a poorly chosen data structure can cost more than the choice of algorithm. Because the heap breaks ties between equal-cost cells in a different order, node counts in the second stage differ slightly from those in Table 5.1, while all path lengths are unchanged.

5.8 Robot Following the Planned Path

Figure 5.7 shows the robot simulation on the 50 × 50 medium-density map. Line-of-sight smoothing reduced the raw A* path to 17 waypoints joined by straight, obstacle-free segments. The robot started at the lower left, followed the smoothed path and reached the goal at the upper right after about 75 s at a cruise speed of 1 cell/s, with its trajectory remaining in free space throughout.

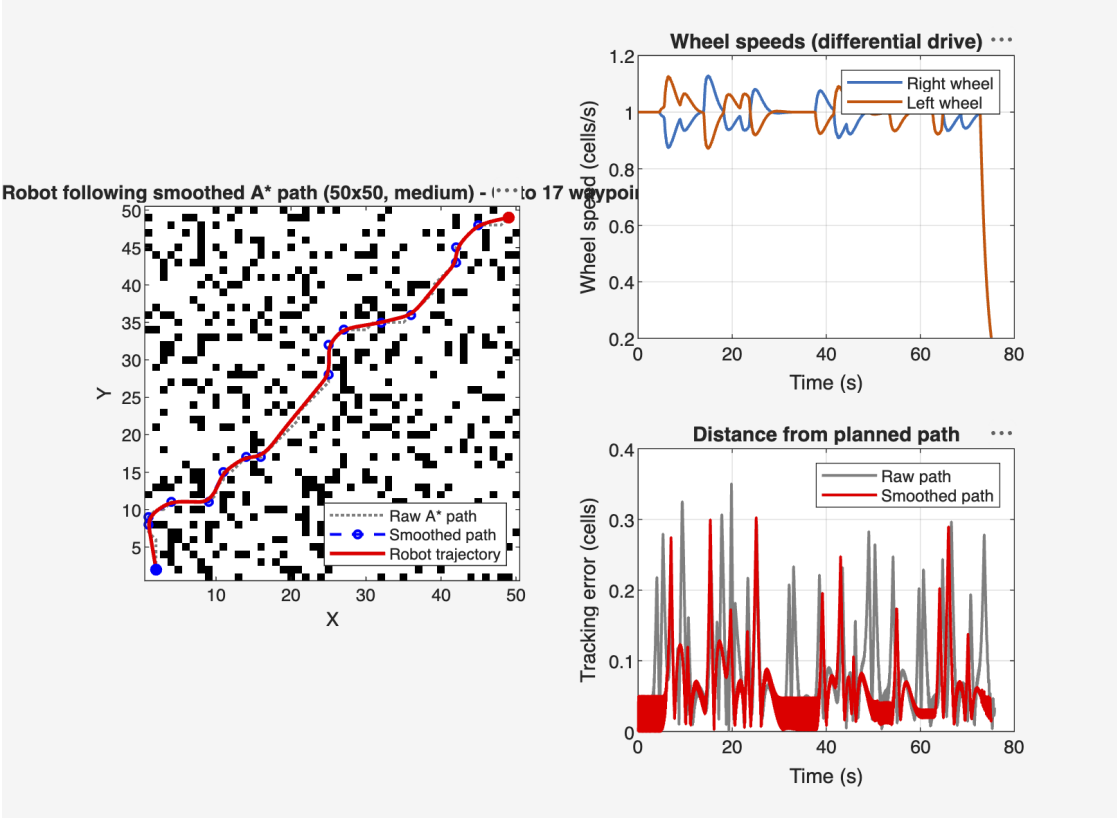


Figure 5.7: Robot motion: raw and smoothed paths with the driven trajectory (left), wheel speeds (top right) and distance from the planned path (bottom right).

The wheel-speed plot shows how a differential-drive robot turns. On straight segments the two wheels run at the same speed; at each waypoint they separate briefly, with the outer wheel running faster, and then rejoin. The drop at the end is the robot slowing down on its approach to the goal. The tracking error stays below about 0.35 cell. Its spikes occur at the turns, where pure pursuit, which aims slightly ahead, cuts the corner a little. Following the smoothed path produced fewer and smaller spikes than following the raw grid path, because the smoothed path has fewer corners to cut.

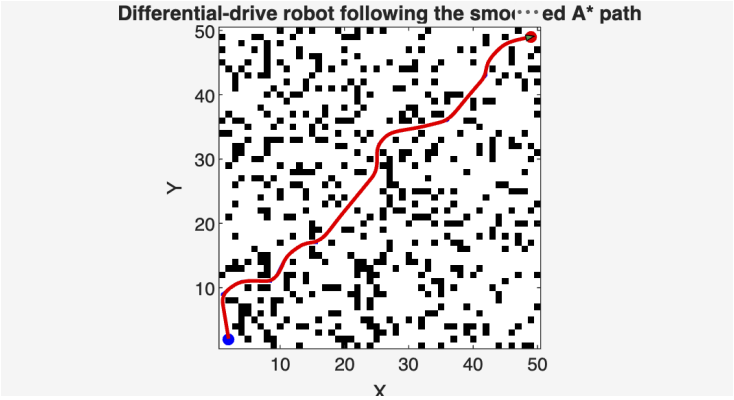


Figure 5.8: Final frame of the animation: the trajectory of the differential-drive robot from start (blue) to goal (red).

5.9 Decision Guideline

The measurements were combined into a simple guideline (Figure 5.9), using A* with the Octile heuristic and the binary-heap open list. A configuration is labelled *either is fine* when Dijkstra already plans in under 1 ms, *A* recommended* when Dijkstra takes longer and A* is up to twice as fast, and *A* strongly recommended* when A* is at least twice as fast.

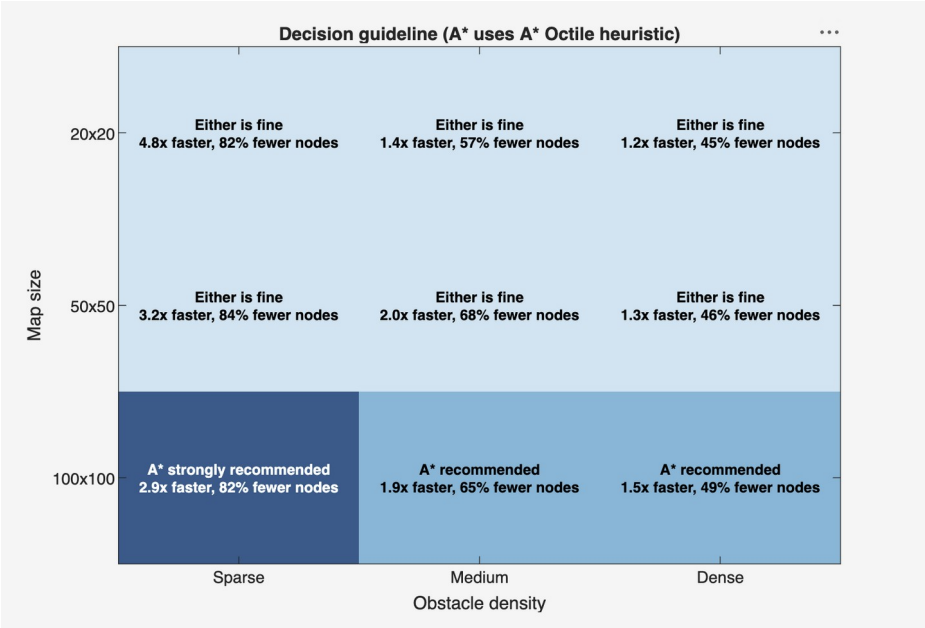


Figure 5.9: Decision guideline for choosing between Dijkstra and A (Octile heuristic, binary-heap open list).*

On the 20 × 20 and 50 × 50 maps both algorithms finish in about a millisecond or less, so the choice makes no practical difference even where A* saves over 80 % of the nodes. The speed-up of 4.8× on the small sparse maps is the ratio of two sub-millisecond times and should not be read as a meaningful advantage. On the 100 × 100 maps A* becomes worthwhile: it is strongly recommended on sparse maps, where it is 2.9× faster with 82 % fewer nodes, and recommended on medium and dense maps, where it is 1.9× and 1.5× faster.

Dijkstra's algorithm still has a role. When a robot needs the distance to many goals at once, for example to choose the nearest of several charging docks, one Dijkstra run provides the cost to every cell, whereas A* would have to be run once per goal.

5.10 Strengths and Limitations

The study has several strengths. Both planners share one function, so the comparison is fair. Every map is seeded and reproducible. Optimality is checked on every map instead of being assumed. Results are reported over 45 maps and four metrics rather than one example, and the effect of the heuristic and of the data structure is measured rather than argued. The robot simulation closes the gap between a list of cells and a motion that a robot could execute.

The limitations are equally clear and lead to the future work of Chapter 6.

- Obstacles are randomly scattered single cells. Real buildings contain walls, corridors and dead ends, where the heuristic can be misled more strongly.
- The maps are static and fully known. Real robots discover obstacles as they move and must replan.
- The robot is treated as a point that fits inside one cell; a real robot needs the obstacles to be inflated by its radius.
- Times were measured in MATLAB on a laptop and are suitable for relative comparison only. Times below 1 ms are dominated by overheads.
- The robot simulation is kinematic and ignores wheel slip, acceleration limits and sensor noise.

5.11 Industrial Relevance

Grid-based planning of the kind studied here is used directly in several industries. Table 5.2 lists areas where the findings apply. In each case a deployed system adds layers that are outside the scope of a laboratory study, such as obstacle inflation, replanning and safety-rated stopping.

Table 5.2: Industrial and research areas related to the project

Area	Link to the project
Warehouse and factory AGVs	Large, structured floors where A* with an exact grid heuristic keeps planning time low as the map grows.
Hospital and office delivery robots	Long corridors with sparse obstacles, the case in which A* gives the largest benefit.
Domestic cleaning robots	Small, cluttered rooms, where either planner is fast enough and coverage matters more than speed.
Charging and docking	Choosing the nearest of several docks, where one Dijkstra run gives costs to all of them.
Robotics education and research	A reproducible benchmark for testing new heuristics and planners against a known baseline.

CHAPTER 6
Conclusion and Future Scope

6.1 Conclusion

This project compared Dijkstra's algorithm and A* for grid-based path planning of an autonomous mobile robot, using a MATLAB benchmark of 45 seeded random maps covering three map sizes and three obstacle densities. Both planners were implemented in one shared search function, and path length, planning time, nodes expanded and memory were logged on every run.

With an admissible heuristic, A* found a path exactly as short as Dijkstra's on every map while expanding 36–78 % fewer nodes. Its advantage depends mainly on obstacle density: it is largest on open maps and shrinks as clutter increases. The Octile heuristic was the best choice for eight-connected motion, expanding about 27 % fewer nodes than Euclidean without losing optimality. The Manhattan heuristic, which overestimates diagonal moves, returned a longer path on 76 % of the maps. Replacing the linear-scan open list with a binary heap made both planners an order of magnitude faster without changing any path. Finally, a simulated differential-drive robot followed the smoothed A* path to the goal using pure-pursuit steering.

The practical recommendation is to use A* with the Octile heuristic and a heap-based open list for grid robots that move in eight directions. On small maps either algorithm is adequate, and Dijkstra remains useful when the distance to many goals is needed at once. For the Robotics and Automation course, the project brought together environment modelling, graph search, algorithm analysis, robot kinematics and path tracking in a single working system.

6.2 Future Scope

- Test on structured maps with walls, rooms and corridors, and on standard benchmark sets [12], to see how much the advantage of A* changes.
- Add dynamic obstacles and replanning, and compare A* with D* Lite [16], which repairs an existing plan instead of starting again.
- Evaluate Jump Point Search [13], Theta* [15] and weighted A* [9] within the same benchmark.
- Inflate obstacles by the robot's radius and add acceleration limits to the motion model.
- Validate the implementation against MATLAB's built-in planners and port the planner to a microcontroller to measure embedded timing.
- Integrate the planner with ROS [5] and run it on a physical robot with a real occupancy map from SLAM.

References

[1] R. Siegwart, I. R. Nourbakhsh and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[2] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[4] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.

[5] M. Quigley *et al.*, "ROS: An open-source Robot Operating System," in *Proc. ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[8] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA, USA: Addison-Wesley, 1984.

[9] I. Pohl, "Heuristic search viewed as path finding in a graph," *Artificial Intelligence*, vol. 1, no. 3–4, pp. 193–204, 1970.

[10] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, Jun. 1989.

[11] S. Thrun, W. Burgard and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[12] N. R. Sturtevant, "Benchmarks for grid-based pathfinding," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 144–148, Jun. 2012.

[13] D. Harabor and A. Grastien, "Online graph pruning for pathfinding on grid maps," in *Proc. 25th AAAI Conf. Artificial Intelligence*, San Francisco, CA, USA, 2011, pp. 1114–1119.

[14] A. Botea, M. Müller and J. Schaeffer, "Near optimal hierarchical path-finding," *J. Game Development*, vol. 1, no. 1, pp. 7–28, 2004.

[15] K. Daniel, A. Nash, S. Koenig and A. Felner, "Theta*: Any-angle path planning on grids," *J. Artificial Intelligence Research*, vol. 39, pp. 533–579, 2010.

[16] S. Koenig and M. Likhachev, "D* Lite," in *Proc. 18th National Conf. Artificial Intelligence (AAAI)*, Edmonton, Canada, 2002, pp. 476–483.

[17] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[18] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Robotics Inst., Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, 1992.

[19] D. Fox, W. Burgard and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, Mar. 1997.

[20] J. W. J. Williams, "Algorithm 232: Heapsort," *Commun. ACM*, vol. 7, no. 6, pp. 347–348, 1964.